

App Configuration: Kconfig

Zephyr RTOS Course





Recap: L02



- `west init` + `west update` — sets up the workspace
- `west.yml` — declares Zephyr version and dependencies
- `west build -b <board>` — builds for a specific board
- `west flash` — programs the binary onto the board



Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



What you'll learn today



- Understand the roles of Kconfig vs Devicetree
- Define and use Kconfig symbols in your project
- Navigate and configure options with `menuconfig`
- Organize options using menus, dependencies, and choices

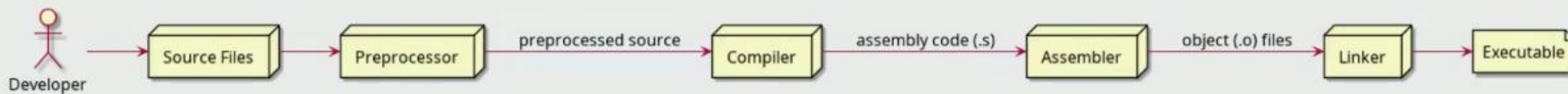


Build and Configuration



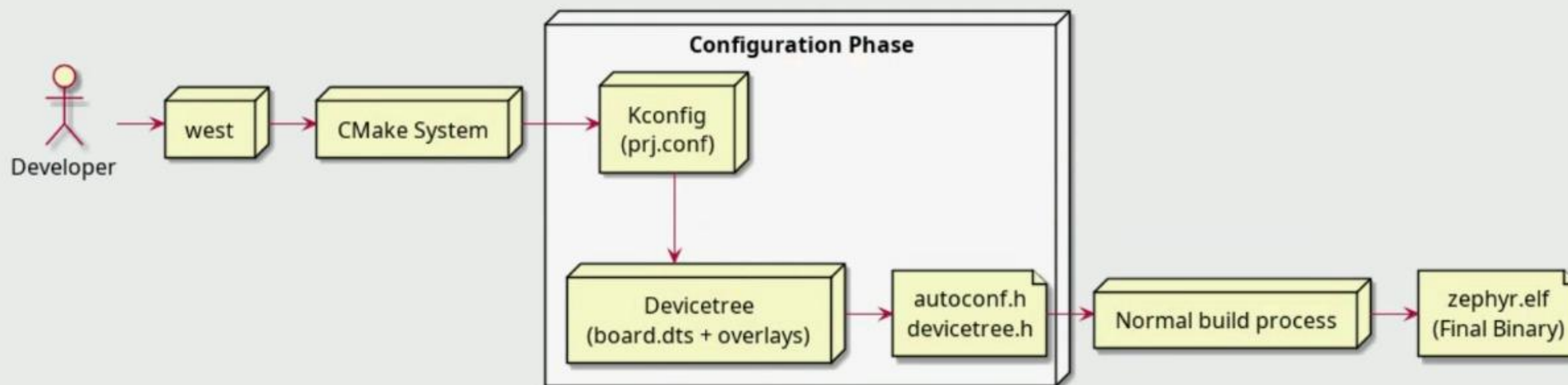


Regular build process



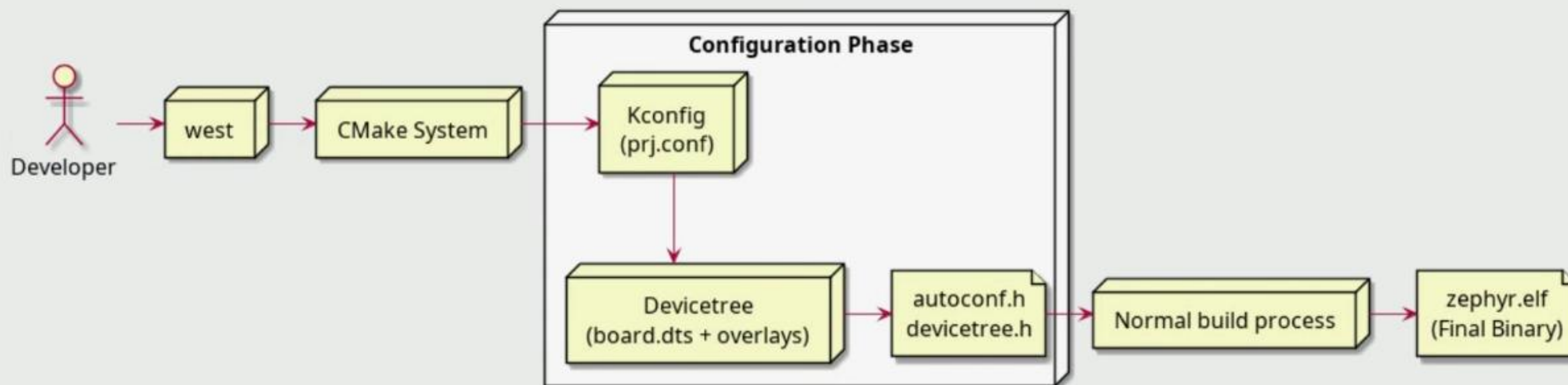


Zephyr build process





Zephyr build process





Zephyr: Two Configuration Systems



	Kconfig	Devicetree
Purpose	Software features	Hardware description
Question	"Do I want this feature?"	"Where is this hardware?"
Example	<code>CONFIG_GPIO=y</code>	<code>gpios = <&gpio0 13></code>
Files	<code>.conf</code> , <code>Kconfig</code>	<code>.dts</code> , <code>.overlay</code>
In code	<code>CONFIG_*</code> macros	<code>DT_*</code> macros



Kconfig + Devicetree = Working App



Example: LED on GPIO pin 13

1. **Devicetree:** "There's an LED on GPIO0 pin 13"

```
gpios = <&gpio0 13 GPIO_ACTIVE_LOW>;
```

2. **Kconfig:** "Enable GPIO driver"

```
CONFIG_GPIO=y
```



Kconfig



What is Kconfig?



Build-time configuration system from Linux kernel

- Enables/disables features **before compilation**
- Controls drivers, subsystems, memory allocation
- User-friendly menu interface (`menuconfig`)



Where is Kconfig stored?



- `prj.conf` — Your project config
- `build/zephyr/.config` — Merged config
- `build/zephyr/include/generated/zephyr/autoconf.h` — C macros (`CONFIG_*`)

Basic Workflow: Edit `prj.conf` → Build → `CONFIG_*` available in code



Demo 1: Control LED Blink Rate



```
config BLINK_SLEEP_TIME_MS
    int "LED blink interval (ms)"
    default 1000
    range 100 5000
    help
        Time between LED toggles in milliseconds.
```

- `int` – type (`bool`, `int`, `string`, `hex`)
- `"LED blink..."` – prompt (user sees this in menuconfig)
- `default 1000` – default value
- `range 100 5000` – validation



Types in Kconfig



Type	Example	Usage
<code>bool</code>	<code>CONFIG_GPIO=y</code>	Enable/disable features
<code>int</code>	<code>CONFIG_BLINK_SLEEP_TIME_MS=2048</code>	Numeric values
<code>string</code>	<code>CONFIG_BT_DEVICE_NAME="Zephyr"</code>	Text values
<code>hex</code>	<code>CONFIG_SRAM_OFFSET=0x1000</code>	Hexadecimal values

Note: `CONFIG_` prefix is **automatically added** in C code!



```
// Boolean
#if CONFIG_GPIO
// or
#ifdef CONFIG_GPIO
// or
if (IS_ENABLED(CONFIG_GPIO)) {} // expands to if (0) or if (1)

// Other types
int delay = CONFIG_BLINK_SLEEP_TIME_MS;
```



Setting values



Edit `prj.conf`:

```
CONFIG_BLINK_SLEEP_TIME_MS=250
```

Use `menuconfig`:

```
west build -t menuconfig # press / to search, S to save, Q to quit
```

Command-line override:

```
west build -- -DCONFIG_BLINK_SLEEP_TIME_MS=2000
```

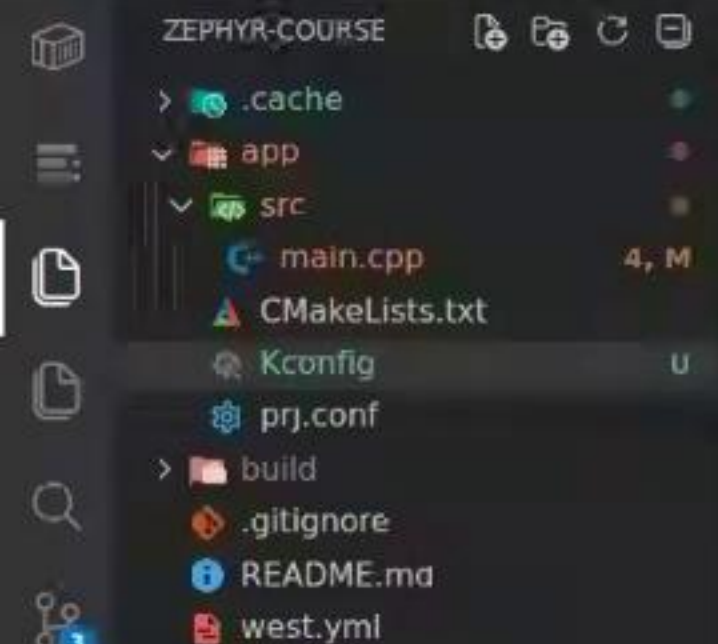
Precedence: menuconfig > CLI > `prj.conf` > default

File Edit Selection View Go Run

zephyr-course

1

🔍 📄 📁 📌 📏 📐 📊 📋 📌 📏 📐 📊 📋



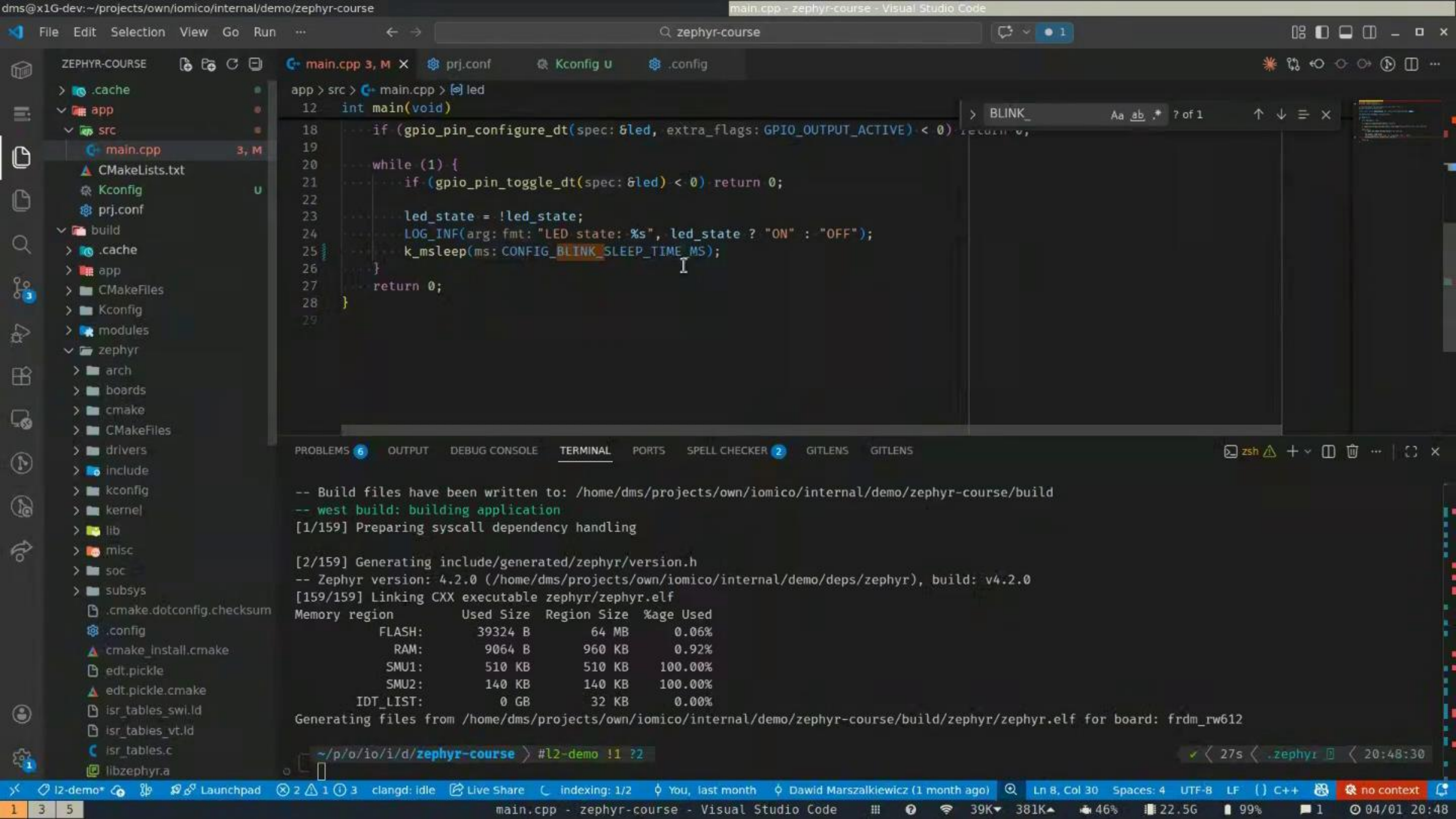
```
main.cpp 4, M prj.conf Kconfig U X
app > Kconfig > [ ] Led subsystem
3     default 1000
4     range 100 5000
5     help
6         help string can be multiline
7         Like this one.
8
9     source "Kconfig.zephyr"
```

> SLEEP_TIME_MS Aa ab . * 1 of 1 ↑ ↓ ≡ ×

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 2 GITLENS GITLENS

python3 ⚠️ + ~ 🗑️ ... 🔍 ×

```
-- ZEPHYR_TOOLCHAIN_VARIANT not set, trying to locate Zephyr SDK
-- Found host-tools: zephyr 0.17.4 (/home/dms/zephyr-sdk-0.17.4)
-- Found toolchain: zephyr 0.17.4 (/home/dms/zephyr-sdk-0.17.4)
-- Found Dtc: /home/dms/zephyr-sdk-0.17.4/sysroots/x86_64-pokysdk-linux/usr/bin/dtc (found suitable version "1.7.0", minimum required is "1.4.6")
-- Found BOARD.dts: /home/dms/projects/own/iomico/internal/demo/deps/zephyr/boards/nxp/frdm_rw612/frdm_rw612.dts
-- Generated zephyr.dts: /home/dms/projects/own/iomico/internal/demo/zephyr-course/build/zephyr/zephyr.dts
-- Generated pickled edt: /home/dms/projects/own/iomico/internal/demo/zephyr-course/build/zephyr/edt.pickle
-- Generated devicetree_generated.h: /home/dms/projects/own/iomico/internal/demo/zephyr-course/build/zephyr/include/generated/zephyr/devicetree_generated.h
Parsing /home/dms/projects/own/iomico/internal/demo/zephyr-course/app/Kconfig
Loaded configuration '/home/dms/projects/own/iomico/internal/demo/deps/zephyr/boards/nxp/frdm_rw612/frdm_rw612_defconfig'
Merged configuration '/home/dms/projects/own/iomico/internal/demo/zephyr-course/app/prj.conf'
Configuration saved to '/home/dms/projects/own/iomico/internal/demo/zephyr-course/build/zephyr/.config'
Kconfig header saved to '/home/dms/projects/own/iomico/internal/demo/zephyr-course/build/zephyr/include/generated/zephyr/autoconf.h'
-- Found GnuLd: /home/dms/zephyr-sdk-0.17.4/arm-zephyr-eabi/arm-zephyr-eabi/bin/ld.bfd (found version "2.38")
-- The C compiler identification is GNU 12.2.0
-- The CXX compiler identification is GNU 12.2.0
-- The ASM compiler identification is GNU
-- Found assembler: /home/dms/zephyr-sdk-0.17.4/arm-zephyr-eabi/bin/arm-zephyr-eabi-gcc
-- Looking for device RW612 in /home/dms/projects/own/iomico/internal/demo/deps/modules/hal/nxp/mcux/mcux-sdk-ng/devices/
-- Found device folder: /home/dms/projects/own/iomico/internal/demo/deps/modules/hal/nxp/mcux/mcux-sdk-ng/devices/Wireless/RW/RW612
-- Found gen_kobject_list: /home/dms/projects/own/iomico/internal/demo/deps/zephyr/scripts/build/gen_kobject_list.py
-- Configuring done (8.6s)
-- Generating done (0.3s)
-- Build files have been written to: /home/dms/projects/own/iomico/internal/demo/zephyr-course/build
-- west build: building application
[1/159] Preparing syscall dependency handling
```





Demo 2: Limit Blink Speeds with Choice



Kconfig: Choice



What is `choice` ?



Select **one option** from multiple alternatives

```
choice
    prompt "LED blink speed"
    default BLINK_SLEEP_1000MS

    config BLINK_SLEEP_250MS
        bool "Fast (250ms)"

    config BLINK_SLEEP_1000MS
        bool "Medium (1s)"

    config BLINK_SLEEP_2000MS
        bool "Slow (2s)"
endchoice
```



Hidden Symbols — Derived Values



```
choice
    prompt "LED blink speed"
    # ... options
endchoice

config BLINK_SLEEP_TIME_MS # use this in code
    int # <-- No prompt = hidden!
    default 250 if BLINK_SLEEP_250MS
    default 1000 if BLINK_SLEEP_1000MS
    default 2000 if BLINK_SLEEP_2000MS
    default 0
```

Purpose: Compute a value based on choice selection



Kconfig: Dependencies & Visibility



depends on

— Conditional availability



```
config LED_BRIGHTNESS
    int "LED brightness (0-100)"
    depends on LED_SUBSYSTEM
    default 100
```

- Symbol is hidden from menu when dependency is `n`
- Cannot be enabled (even manually) if dependency unmet
- User must enable `LED_SUBSYSTEM` first



`if` block — Bulk dependency



```
if LED_SUBSYSTEM

config LED_BRIGHTNESS # Automatically gets: depends on LED_SUBSYSTEM
    int "LED brightness (0-100)"

config LED_FADE_DURATION_MS # Also gets: depends on LED_SUBSYSTEM
    int "Fade duration (ms)"

endif # LED_SUBSYSTEM
```

- Wraps multiple symbols — all inherit `depends on`
- Cleaner when many symbols share the same dependency



range

— Value validation



```
config LED_BRIGHTNESS
    int "LED brightness (0-100)"
    range 0 100
    default 100
```

```
config LED_FADE_DURATION_MS
    int "Fade duration (ms)"
    range 0 5000
    default 500
```

- Build fails if value is outside range
- Works with `int` and `hex` types



Kconfig: Organization



menu

— Group options together



```
menu "LED settings"

config LED_BRIGHTNESS
    int "LED brightness (0-100)"

config LED_FADE_DURATION_MS
    int "Fade duration (ms)"

endmenu
```

- Pure visual container — no config symbol created
- Shows as a submenu entry: LED settings --->



`visible if`

— Hide menu, keep symbols



```
menu "Expert settings"
  visible if LED_ADVANCED

config LED_DEBUG
  bool "Enable LED debugging"
  # Does NOT get depends on LED_ADVANCED!
endmenu
```



`if` block vs `visible if`



	<code>if FEAT</code>	<code>visible if FEAT</code>
Adds <code>depends on</code>	✓	✗
Hides from menu	✓	✓
Symbol settable when hidden	✗	✓

Rule of thumb:

- `if` block — symbols **shouldn't work** without the dependency
- `visible if` — symbols **can work**, just hide the menu for clarity



menuconfig

— Checkbox + submenu



```
menuconfig LED_SUBSYSTEM
    bool "LED Subsystem"
    default y

if LED_SUBSYSTEM

config LED_BRIGHTNESS
    int "LED brightness (0-100)"
    default 100

endif # LED_SUBSYSTEM
```



`menuconfig` — Checkbox + submenu



- Creates a bool symbol `LED_SUBSYSTEM` **and** a submenu
- When unchecked, all children are hidden and disabled
- `menu` = folder, `menuconfig` = folder with an on/off switch

In `menuconfig`:

```
[*] LED Subsystem ---> # menuconfig (toggleable)
    LED settings  ---> # menu (always visible)
```



Homework



Homework: Design a Hierarchical Menu



Reproduce this menu structure in your `Kconfig`:

```
[*] LED Subsystem --->
    LED blink sleep time (1s (medium)) --->
    [ ] Advanced LED settings --->
        (100) LED brightness (0-100)
        (500) LED fade duration (ms)
        Expert settings --->
            [ ] Enable LED debugging
            [ ] Custom blink pattern
```

Push tag `l3-task1`



Requirements



- `menuconfig` for LED Subsystem (top-level toggle)
- `choice` for blink sleep time (250ms, 500ms, 1s, 2s) with hidden int symbol
- `menuconfig` for Advanced LED settings
- `menu` with `visible if` for Expert settings
- `range` for brightness (0-100) and fade (0-5000)

Thank You!

Questions?

